

```

#!/usr/bin/env python3
"""
OTR News builder.
Pulls trucking headlines from feeds.txt, merges them into a rolling archive
(data/items.json), adds our own articles from posts/, and writes the site to ./si
Standard library only.
"""

import base64
import html
import json
import re
import sys
import urllib.parse
import urllib.request
import xml.etree.ElementTree as ET
from datetime import datetime, timedelta, timezone
from email.utils import parsedate_to_datetime, format_datetime
from pathlib import Path

# ===== SETTINGS: edit these =====
SITE_URL = "https://otrnews.com"
SITE_NAME = "OTR News"
TAGLINE = "Trucking news for owner-operators, small fleets, and drivers."
CONTACT_EMAIL = "" # e.g. "news@otrnews.com" - turns on the Contact page
NEWSLETTER_URL = "" # e.g. your Beehiiv signup link - turns on signup boxes
# =====

KEEP_DAYS = 30 # how long outside headlines stay in the archive
ON_PAGE = 100 # outside headlines on the homepage
ORIGINALS_ON_PAGE = 8 # our own articles leading the homepage

ROOT = Path(__file__).parent
SITE = ROOT / "site"
ARCHIVE = ROOT / "data" / "items.json"
POSTS = ROOT / "posts"
PAGES = ROOT / "pages"
UA = "Mozilla/5.0 (compatible; OTRNewsBot/1.0; +https://otrnews.com)"

# First match wins, so order matters.
CATEGORIES = [
    ("Regulations", r"\b(fmcsa|eld|elds|hours.of.service|hos|rulemaking|rule|rule
    ("Fuel", r"\b(diesel|fuel|gas prices?|opec|crude|refiner\w*|oil prices?)\b"),
    ("Enforcement & safety", r"\b(crash\w*|rollover|fatal\w*|collision|safety|sei
    ("Freight market", r"\b(freight|spot rates?|contract rates?|load posts?|load

```

```

    ("Equipment", r"\b(trailers?|engines?|electric trucks?|ev|evs|autonomous|driv
    ("Drivers", r"\b(drivers?|truckers?|owner.operators?|cdl|cdls|truck parking|p
    ("Business", r"\b(bankrupt\w*|acquisitions?|acquires?|mergers?|layoffs?|earn
]

```

```

IMAGES = {'og.png': 'ivBORw0KGgoAAAAANSUhEUgAABLA AAAJ2CAMAAAB4notuAAAAAYFBMVEX///8

```

```

# ----- fetching & parsing -----

```

```

def read_feeds():
    feeds = []
    for line in (ROOT / "feeds.txt").read_text().splitlines():
        line = line.strip()
        if not line or line.startswith("#") or "|" not in line:
            continue
        name, url = [p.strip() for p in line.split("|", 1)]
        feeds.append((name, url))
    return feeds

```

```

def fetch(url):
    req = urllib.request.Request(url, headers={"User-Agent": UA, "Accept": "appli
    with urllib.request.urlopen(req, timeout=25) as r:
        return r.read()

```

```

def strip_ns(tag):
    return tag.rsplit("}", 1)[-1].lower()

```

```

def child_text(el, *names):
    for c in el:
        if strip_ns(c.tag) in names and (c.text or "").strip():
            return c.text.strip()
    return ""

```

```

def parse_date(s):
    if not s:
        return None
    try:
        d = parsedate_to_datetime(s)
    except (TypeError, ValueError):
        try:
            d = datetime.fromisoformat(s.replace("Z", "+00:00"))
        except ValueError:

```

```

        return None
    if d.tzinfo is None:
        d = d.replace(tzinfo=timezone.utc)
    return d.astimezone(timezone.utc)

def clean(text, limit=None):
    text = html.unescape(re.sub(r"<[^>]+>", " ", text or ""))
    text = re.sub(r"\s+", " ", text).strip()
    text = re.sub(r"(The post .*? appeared first on .*?\.?.$)", "", text).strip()
    if limit and len(text) > limit:
        text = text[:limit].rsplit(" ", 1)[0].rstrip(",.;:") + "..."
    return text

def parse_feed(source, raw):
    root = ET.fromstring(raw)
    items = []
    for el in root.iter():
        kind = strip_ns(el.tag)
        if kind not in ("item", "entry"):
            continue
        title = clean(child_text(el, "title"))
        link = child_text(el, "link")
        if not link: # Atom
            for c in el:
                if strip_ns(c.tag) == "link" and c.get("href") and c.get("rel", ""):
                    link = c.get("href")
                    break
        date = parse_date(child_text(el, "pubdate", "published", "updated", "date"))
        summary = clean(child_text(el, "description", "summary", "encoded"), 240)
        src = source
        if "news.google.com" in (link or ""):
            publisher = child_text(el, "source")
            if publisher:
                src = publisher
                title = re.sub(r"\s+--\s+" + re.escape(publisher) + r"$", "", title)
                summary = "" # Google's summaries just repeat the headline
        if not title or not link or not link.startswith("http"):
            continue
        items.append({
            "title": title,
            "link": link,
            "source": src,
            "summary": summary,
            "published": (date or datetime.now(timezone.utc)).isoformat(),
        })

```

```

    return items

def categorize(item):
    text = (item["title"] + " " + item.get("summary", "")).lower()
    for name, pattern in CATEGORIES:
        if re.search(pattern, text):
            return name
    return "Industry"

def key_for(item):
    return re.sub(r"^[a-z0-9]", "", item["title"].lower())[:80]

# ----- archive -----

def load_archive():
    if ARCHIVE.exists():
        try:
            return json.loads(ARCHIVE.read_text())
        except json.JSONDecodeError:
            pass
    return []

def merge(old, new, allowed_sources):
    by_key = {}
    for i in old + new:
        if i.get("original") or i.get("source") not in allowed_sources:
            continue # drops sources you've switched off in feeds.txt
        by_key.setdefault(key_for(i), i)
    cutoff = datetime.now(timezone.utc) - timedelta(days=KEEP_DAYS)
    now = datetime.now(timezone.utc) + timedelta(hours=1)
    items = [i for i in by_key.values() if cutoff <= datetime.fromisoformat(i["pu
    for i in items:
        i["category"] = categorize(i)
    items.sort(key=lambda i: i["published"], reverse=True)
    return items

# ----- markdown -----

def esc(s):
    return html.escape(s or "", quote=True)

```

```

def slugify(s):
    return re.sub(r"^[a-z0-9]+", "-", s.lower()).strip("-")[:80]

def inline_md(t):
    t = esc(t)
    t = re.sub(r"\*\*(.+?)\*\*", r"<strong>\1</strong>", t)
    t = re.sub(r"(?![\w*])\*(?!\\s)(.+?) (?!\\s)\*(?![\w*])", r"<em>\1</em>", t)
    def link(m):
        url = m.group(2)
        ext = not url.startswith("/") and not url.startswith("mailto:")
        return f'<a href="{url}"' + (' target="_blank" rel="noopener"' if ext else '')
    t = re.sub(r"\[[^\]]+\]\(\s*(https?://[^\s]+|/[^\s]*|mailto:[^\s]+)\s*\)", link, t)
    return t

def markdown(md):
    out, para, lst = [], [], None
    def flush():
        nonlocal para, lst
        if para:
            out.append("<p>" + inline_md(" ".join(para)) + "</p>"); para = []
        if lst:
            out.append("<ul>" + "".join(f"<li>{inline_md(x)}</li>" for x in lst))
    for line in md.splitlines():
        s = line.strip()
        if not s:
            flush(); continue
        if s.startswith("### "):
            flush(); out.append(f"<h3>{inline_md(s[4:])}</h3>")
        elif s.startswith("## "):
            flush(); out.append(f"<h2>{inline_md(s[3:])}</h2>")
        elif s[:2] in ("- ", "* "):
            if para: flush()
            lst = (lst or []) + [s[2:]]
        else:
            if lst: flush()
            para.append(s)
    flush()
    return "\n".join(out)

def split_front_matter(text):
    meta, body = {}, text
    if text.startswith("---"):
        _, head, body = text.split("---", 2)

```

```

        for line in head.strip().splitlines():
            if ":" in line:
                k, v = line.split(":", 1)
                v = v.strip()
                if len(v) >= 2 and v[0] == v[-1] and v[0] in "\"'":
                    v = v[1:-1]
                meta[k.strip().lower()] = v
    return meta, body

# ----- our own articles (posts/*.md) -----

def load_posts():
    posts = []
    if not POSTS.exists():
        return posts
    for f in sorted(POSTS.glob("*.md")):
        meta, body = split_front_matter(f.read_text(encoding="utf-8"))
        if meta.get("draft", "").lower() == "true" or not meta.get("title"):
            continue
        date = parse_date(meta.get("date", "")) or datetime.fromtimestamp(f.stat(
            slug = meta.get("slug") or slugify(re.sub(r"^\d{4}-\d{2}-\d{2}-", "", f.s
            words = len(re.findall(r"\w+", body))
        posts.append({
            "title": meta["title"], "link": f"/news/{slug}/", "slug": slug,
            "source": SITE_NAME, "author": meta.get("author", "OTR News Staff"),
            "summary": meta.get("summary", ""), "category": meta.get("category")
            "published": date.isoformat(), "original": True,
            "minutes": max(1, round(words / 230)), "body": markdown(body),
        })
    posts.sort(key=lambda p: p["published"], reverse=True)
    return posts

# ----- default pages (override by adding pages/about.md etc.) -----

def default_pages():
    contact_line = (f"Send news tips, corrections, and advertising questions to [
        if CONTACT_EMAIL else ""
    )
    pages = {
        "about": ("About OTR News", "Who we are and how we report.", f"""
OTR News covers the trucking industry for the people who keep it moving: owner-op

## What we cover

Regulations and enforcement, freight rates, fuel, equipment, and the business of

```

How we report

Every OTR News article is researched from primary sources, like FMCSA, DOT, the F

We use AI tools to help research and draft articles. An editor reviews every stor

Around the industry

Our homepage also links to headlines from other trucking news outlets. Those link

```
{contact_line}
```

```
"""),
```

```
    "privacy": ("Privacy policy", "How OTR News handles information.", f""
```

```
*Last updated {datetime.now(timezone.utc):%B %-d, %Y}*
```

OTR News is a news website. You can read everything on it without creating an acc

What we collect

We don't ask for your name, email, or any other personal information to read the

Services we rely on

- **Hosting.** The site is hosted on GitHub Pages. Like most web hosts, GitHub ma
 - **Fonts.** The site loads its typeface from Google Fonts, which means your brow
 - **Links to other sites.** Headlines and sources link to other publishers, whose
- {- **Newsletter.** If you sign up for our newsletter, your email address is hand

Changes

If we add features like a newsletter, analytics, or advertising, we'll update thi

```
(("## Contact" + chr(10) + chr(10) + f"Questions about this policy: [{CONTACT_EMA
```

```
"""),
```

```
}
```

```
    if CONTACT_EMAIL:
```

```
        pages["contact"] = ("Contact OTR News", "Send us news tips, corrections,
```

```
We want to hear from drivers, owner-operators, and fleet owners.
```

- **News tips.** Seeing something on the road, at a shipper, or in your inbox tha
- **Corrections.** If we got something wrong, let us know and we'll fix it.
- **Advertising.** Reach owner-operators and small fleets. Ask us about sponsorsh

```
Email: [{CONTACT_EMAIL}] (mailto:{CONTACT_EMAIL})
```

```
""")
```

```
    return pages
```

```

def load_pages():
    pages = default_pages()
    if PAGES.exists():
        for f in PAGES.glob("*.md"):
            meta, body = split_front_matter(f.read_text(encoding="utf-8"))
            name = f.stem.lower()
            old = pages.get(name, (name.title(), "", ""))
            pages[name] = (meta.get("title", old[0]), meta.get("summary", old[1]))
    return pages

```

```

# ----- shared page shell -----

```

```

def css_from(tpl):
    return re.search(r"<style>.*?</style>", tpl, re.S).group(0)

```

```

def footer_links(pages):
    links = [("/about/", "About")]
    if "contact" in pages:
        links.append("/contact/", "Contact")
    links += [("/privacy/", "Privacy"), ("/feed.xml", "RSS")]
    return "&nbsp;".join(f'<a href="{u}">{t}</a>' for u, t in links)

```

```

def newsletter_box():
    if not NEWSLETTER_URL:
        return ""
    return f"""<aside class="signup">
<h2>Get the morning briefing</h2>
<p>The trucking news that matters to small carriers, in your inbox before you rol
<a class="btn" href="{esc(NEWSLETTER_URL)}" target="_blank" rel="noopener">Sign u
</aside>"""

```

```

PAGE_CSS = """<style>
.mast.small{padding-top:1.25rem}
.mast.small .sign-inner{padding:.8rem 1.1rem .7rem}
.mast.small .logo{font-size:2rem}
.doc{padding-top:1.5rem;padding-bottom:2rem}
.doc h1{font-weight:900;font-size:clamp(1.9rem,6.5vw,2.8rem);line-height:1.1;marg
.doc .deck{font-size:1.2rem;color:var(--muted);margin:0 0 1rem}
.byline{font-size:.9rem;color:var(--muted);padding-bottom:1rem;border-bottom:1px
.body p,.body li{font-size:1.1rem;line-height:1.65}
.body h2{font-weight:800;font-size:1.45rem;margin:2rem 0 .4rem}
.body h3{font-weight:800;font-size:1.2rem;margin:1.5rem 0 .3rem}

```

```
.body a{color:var(--sign);text-underline-offset:2px}
@media (prefers-color-scheme:dark){.body a{color:#5CC795}}
.share{display:flex;flex-wrap:wrap;gap:.5rem;margin:2rem 0 0;padding-top:1.25rem;
.share span{width:100%;font-weight:800;margin-bottom:.1rem}
.share a,.share button{font:600 .95rem var(--font);color:var(--ink);background:va
.more{margin-top:2.5rem}
.more h2{font-weight:900;font-size:1.3rem;margin:0 0 .25rem;padding-top:1rem;bord
.back{display:inline-block;margin-top:1.5rem;font-weight:700}
</style>"""
```

```
def shell(tpl, pages, *, title, description, path, body, og_type="website", ld=No
    ld_html = f'<script type="application/ld+json">{json.dumps(ld)}</script>' if
    return f'""<!doctype html>
<html lang="en"><head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1, viewport-fit=
<title>{esc(title)} | {SITE_NAME}</title>
<meta name="description" content="{esc(description)}">
<link rel="canonical" href="{SITE_URL}{path}">
<meta property="og:site_name" content="{SITE_NAME}">
<meta property="og:title" content="{esc(title)}">
<meta property="og:description" content="{esc(description)}">
<meta property="og:type" content="{og_type}">
<meta property="og:url" content="{SITE_URL}{path}">
<meta property="og:image" content="{SITE_URL}/og.png">
<meta name="twitter:card" content="summary_large_image">
<meta name="theme-color" content="#00603C">
<link rel="icon" href="/icon.png"><link rel="apple-touch-icon" href="/icon.png">
<link rel="alternate" type="application/rss+xml" title="{SITE_NAME}" href="/feed.
<link rel="preconnect" href="https://fonts.googleapis.com"><link rel="preconnect"
<link href="https://fonts.googleapis.com/css2?family=Overpass:wght@400;600;800;90
{ld_html}
{css_from(tpl)}
{PAGE_CSS}
</head><body>
<header class="mast small wrap"><div class="sign"><div class="sign-inner"><p clas
<main class="wrap doc">
{body}
</main>
<footer class="wrap"><p>{footer_links(pages)}</p><p>&copy; {datetime.now(timezone
</body></html>"""
```

```
def render_article(p, posts, tpl, pages):
    d = datetime.fromisoformat(p["published"])
    url = SITE_URL + p["link"]
```

```

q = urllib.parse.quote
share = f"""<div class="share"><span>Share this story</span>
<a href="https://www.facebook.com/sharer/sharer.php?u={q(url, safe='')}" target="
<a href="https://twitter.com/intent/tweet?url={q(url, safe='')}&text={q(p['title'
<a href="mailto:?subject={q(p['title'], safe='')}&body={q(url, safe='')}">Email</
<button type="button" onclick="navigator.clipboard&&navigator.clipboard.writeText
</div>"""

others = [o for o in posts if o is not p][:4]
more = ""
if others:
    more = '<section class="more"><h2>More from OTR News</h2>' + "".join(stor
    body = f"""<article>
<p class="meta"><span class="cat">{esc(p['category'])}</span></p>
<h1>{esc(p['title'])}</h1>
<p class="deck">{esc(p['summary'])}</p>
<p class="byline"><span>By {esc(p['author'])}</span><time datetime="{p['published
<div class="body">
{p['body']}
</div>
{share}
</article>
{newsletter_box()}
{more}
<a class="back" href="/">All trucking news</a>"""

ld = {"@context": "https://schema.org", "@type": "NewsArticle", "headline": p
    "datePublished": p["published"], "dateModified": p["published"],
    "author": {"@type": "Organization", "name": p["author"]},
    "publisher": {"@type": "Organization", "name": SITE_NAME,
        "logo": {"@type": "ImageObject", "url": f"{SITE_URL}/icon
    "image": [f"{SITE_URL}/og.png"], "description": p["summary"], "mainEnti
return shell(tpl, pages, title=p["title"], description=p["summary"], path=p["
    body=body, og_type="article", ld=ld)

def render_page(name, page, tpl, pages):
    title, desc, md = page
    body = f'<h1>{esc(title)}</h1>\n<div class="body">\n{markdown(md)}\n</div>\n<
    return shell(tpl, pages, title=title, description=desc or title, path=f"/{nam

# ----- homepage -----

def fmt_date(iso):
    return datetime.fromisoformat(iso).strftime("%b %-d, %Y")

def story_html(i, lead=False):

```

```

        cls = ("story lead" if lead else "story") + (" original" if i.get("original")
        summary = f'<p class="dek">{esc(i["summary"])}</p>' if i.get("summary") else
        tag = "h2" if lead else "h3"
        target = "" if i.get("original") else ' target="_blank" rel="noopener"'
        return f'<article class="{cls}" data-cat="{esc(i["category"])}">
<p class="meta"><span class="cat">{esc(i["category"])}</span><span class="src">
<{tag}><a href="{esc(i["link"])}" {target}>{esc(i["title"])}</a></{tag}>
{summary}
</article>'"""

```

```

def render(items, originals, sources_ok, pages):
    updated = datetime.now(timezone.utc)
    originals = originals[:ORIGINALS_ON_PAGE]
    feed = items[:ON_PAGE]
    cats = [c for c, _ in CATEGORIES] + ["Industry"]
    present = [c for c in cats if any(i["category"] == c for i in feed)]
    tpl = (ROOT / "template.html").read_text()
    chips = "".join(f'<button type="button" class="chip" data-filter="{esc(c)}" a
if originals:
    ours = story_html(originals[0], True) + "\n".join(story_html(i) for i in
    industry_intro = '<h2 class="section-title" id="industry">Around the indu
    feed_html = "\n".join(story_html(i) for i in feed)
else:
    ours, industry_intro = "", ""
    feed_html = (story_html(feed[0], True) + "\n".join(story_html(i) for i in
        '<p class="empty">No stories yet. The next update will fill this page
ld = {"@context": "https://schema.org", "@type": "WebSite", "name": SITE_NAME
return (tpl.replace("{{SITE_NAME}}", esc(SITE_NAME))
        .replace("{{TAGLINE}}", esc(TAGLINE))
        .replace("{{SITE_URL}}", SITE_URL)
        .replace("{{UPDATED_ISO}}", updated.isoformat())
        .replace("{{UPDATED_TEXT}}", updated.strftime("%b %-d, %Y at %H:%M
        .replace("{{SOURCE_COUNT}}", str(sources_ok))
        .replace("{{CHIPS}}", chips)
        .replace("{{ORIGINALS}}", ours)
        .replace("{{NEWSLETTER}}", newsletter_box())
        .replace("{{INDUSTRY_TITLE}}", industry_intro)
        .replace("{{STORIES}}", feed_html)
        .replace("{{FOOTER_LINKS}}", footer_links(pages))
        .replace("{{JSONLD}}", json.dumps(ld))
        .replace("{{YEAR}}", str(updated.year))

```

```

def render_rss(entries):
    out = []
    for i in entries[:50]:

```

```

        d = format_datetime(datetime.fromisoformat(i["published"]))
        link = SITE_URL + i["link"] if i["link"].startswith("/") else i["link"]
        out.append(f"<item><title>{esc(i['title'])}</title><link>{esc(link)}</link><description>{i['description']}</description></item>")
    now = format_datetime(datetime.now(timezone.utc))
    return f'<?xml version="1.0" encoding="UTF-8"?>
<rss version="2.0"><channel><title>{SITE_NAME}</title><link>{SITE_URL}</link><description>{"".join(out)}</channel></rss>'

```

```

def main():
    feeds = read_feeds()
    fresh, ok = [], 0
    for name, url in feeds:
        try:
            got = parse_feed(name, fetch(url))
            fresh.extend(got)
            ok += 1
            print(f"    ok    {name}: {len(got)} stories")
        except Exception as e:
            print(f"    skip {name}: {e.__class__.__name__}: {e}", file=sys.stderr)

    items = merge(load_archive(), fresh, {n for n, _ in feeds})
    ARCHIVE.parent.mkdir(exist_ok=True)
    ARCHIVE.write_text(json.dumps(items, indent=1))

    SITE.mkdir(exist_ok=True)
    tpl = (ROOT / "template.html").read_text()
    pages = load_pages()
    posts = load_posts()
    for p in posts:
        p["category"] = p["category"] or categorize(p)
    for p in posts:
        out = SITE / "news" / p["slug"]
        out.mkdir(parents=True, exist_ok=True)
        (out / "index.html").write_text(render_article(p, posts, tpl, pages))
    for name, page in pages.items():
        out = SITE / name
        out.mkdir(parents=True, exist_ok=True)
        (out / "index.html").write_text(render_page(name, page, tpl, pages))
    (SITE / "404.html").write_text(shell(tpl, pages, title="Page not found", desc=
        body='<h1>That page isn\'t here</h1><div class="body"><p>It may have move
for name, data in IMAGES.items():
    (SITE / name).write_bytes(base64.b64decode(data))

(SITE / "index.html").write_text(render(items, posts, ok, pages))
everything = sorted(posts + items, key=lambda i: i["published"], reverse=True)

```

```

(SITE / "feed.xml").write_text(render_rss(everything))
(SITE / "robots.txt").write_text(f"User-agent: *\nAllow: /\nSitemap: {SITE_URL}
urls = [f"<url><loc>{SITE_URL}</loc><changefreq>hourly</changefreq></url>"]
urls += [f'<url><loc>{SITE_URL}</loc><lastmod>{p["published"][:10]}</url>']
urls += [f"<url><loc>{SITE_URL}</loc></url>" for n in pages]
(SITE / "sitemap.xml").write_text('<?xml version="1.0" encoding="UTF-8"?><url
print(f"Built site: {len(posts)} articles, {len(items)} headlines in archive,

if __name__ == "__main__":
    main()

```